Mohamed REKIK
Clément LITAIZE-RICHERT M1 EEEA

Microcontrôleur – Communication CAN et UART

1- Introduction :

Dans cette partie du projet, l'objectif est de mettre en œuvre une communication entre deux systèmes embarqués à l'aide du bus CAN (Controller Area Network), puis de transmettre les données reçues vers l'ordinateur via une liaison UART.

Le système est composé de :

Une carte Arduino équipée d'un shield CAN : MCP2515 qui émet des trames CAN

Une carte STM32F407GTx Discovery qui reçoit ces trames CAN

Un PC qui reçoit les données via une interface USB–UART : FT232RL

Le microcontrôleur STM32 joue donc le rôle de passerelle CAN → UART.

2- Logiciels et outils utilisés :

Les outils suivants ont été utilisés pour le développement :

STM32CubeMX il est utilisé pour :

-Configurer l'horloge du microcontrôleur

-Configurer les périphériques CAN et UART pour assurer les communications

-Générer automatiquement le code d'initialisation

Keil uVision utilisé pour :

-Compléter le code C

-Compiler et programmer la carte STM32

-Déboguer l'application

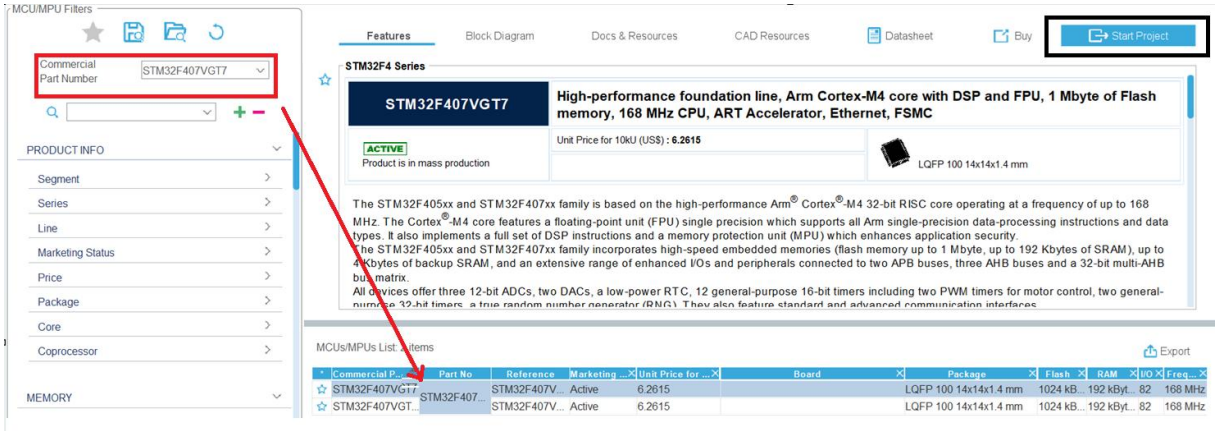
Arduino IDE utilisé pour :

Programmer l'Arduino

Envoyer des trames CAN via le shield MCP2515

3- Génération du projet MKD STM32 (Réponse à la question 1)

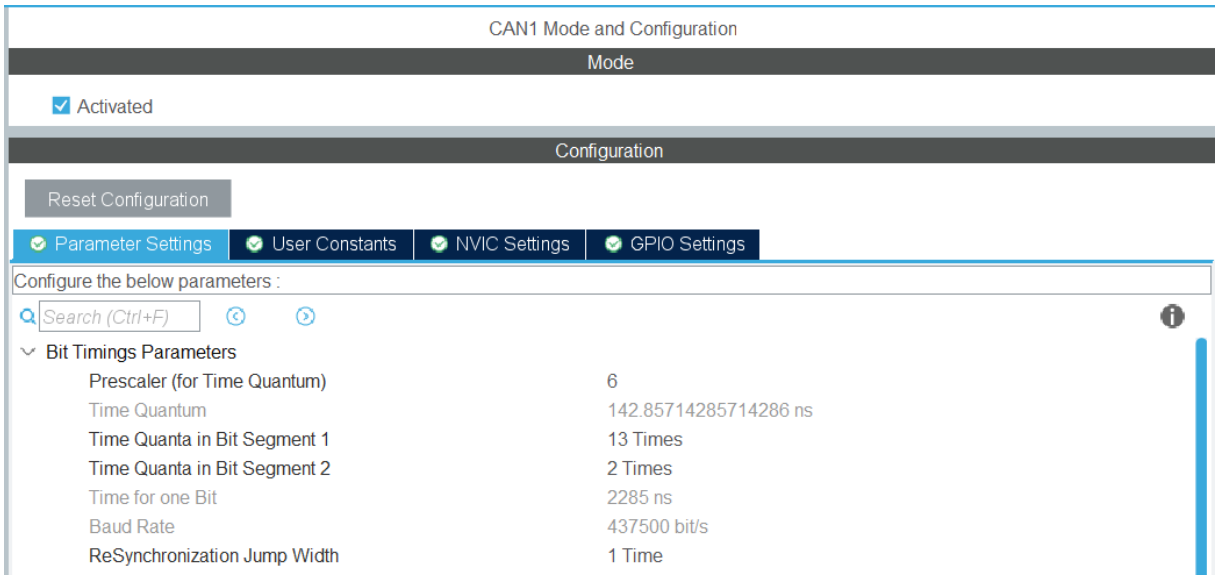
Le projet STM32 a été généré à l'aide de STM32CubeMX. Au démarrage de ce dernier, on trouve les cartes microcontrôleur : l'installation des drivers est obligatoire pour le fonctionnement correct et surtout l'exportation au logiciel Keil pour compléter la liaison CARTE-PC.



Dès qu'on finit de choisir notre carte du microcontrôleur, on clique deux fois dessous ou indique la flèche et **« Start Project »** encadré en haut en noir

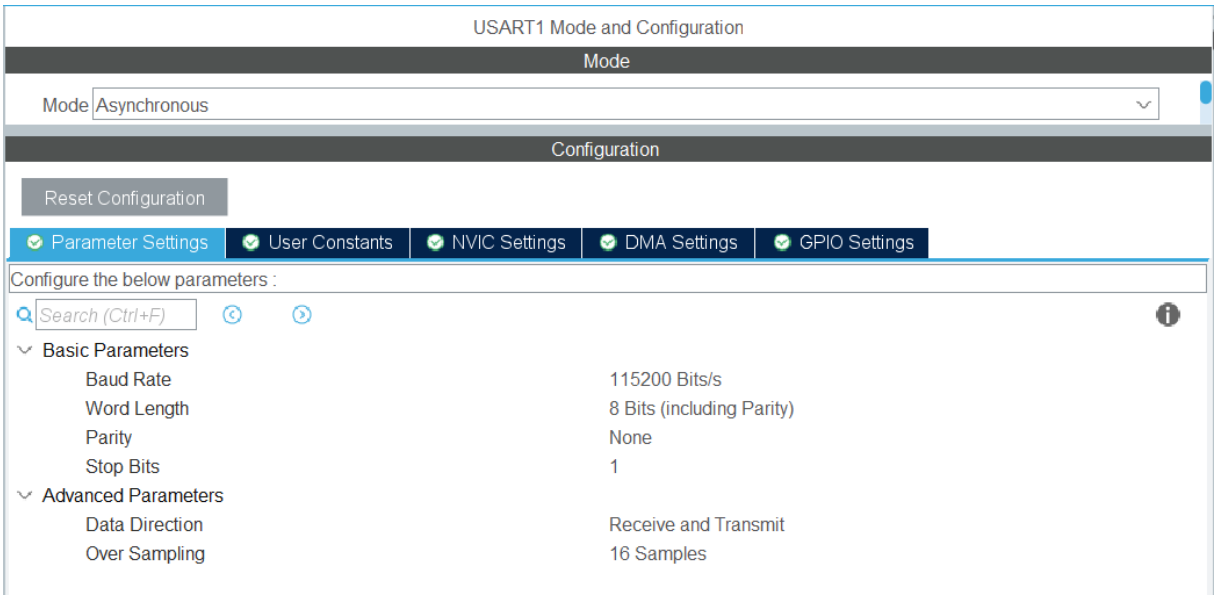
Les périphériques suivants ont été activés :

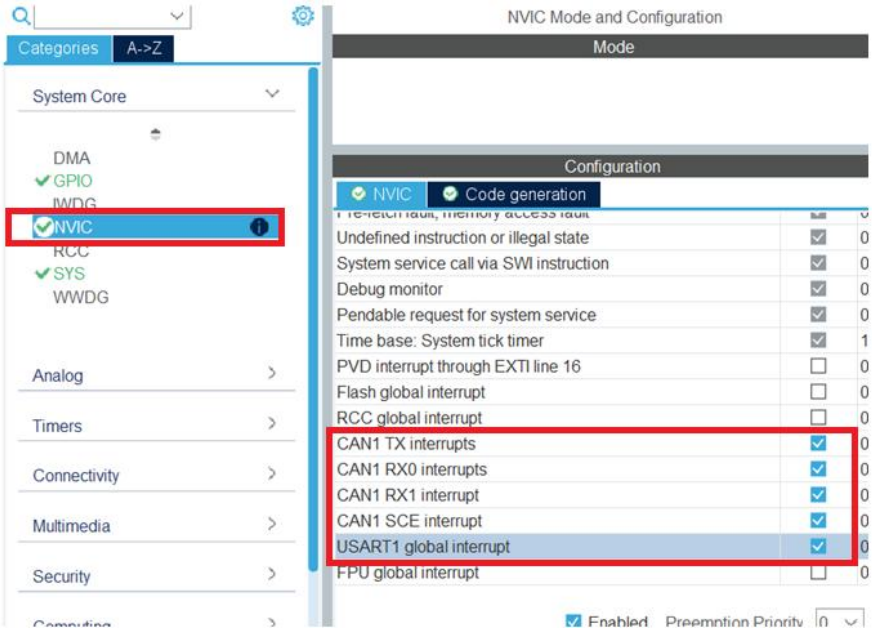
CAN1 en mode normal



Ces paramètres ont été choisis afin d'obtenir une communication CAN stable et compatible avec l'Arduino utilisant un contrôleur MCP2515 configuré à 500 kbps.

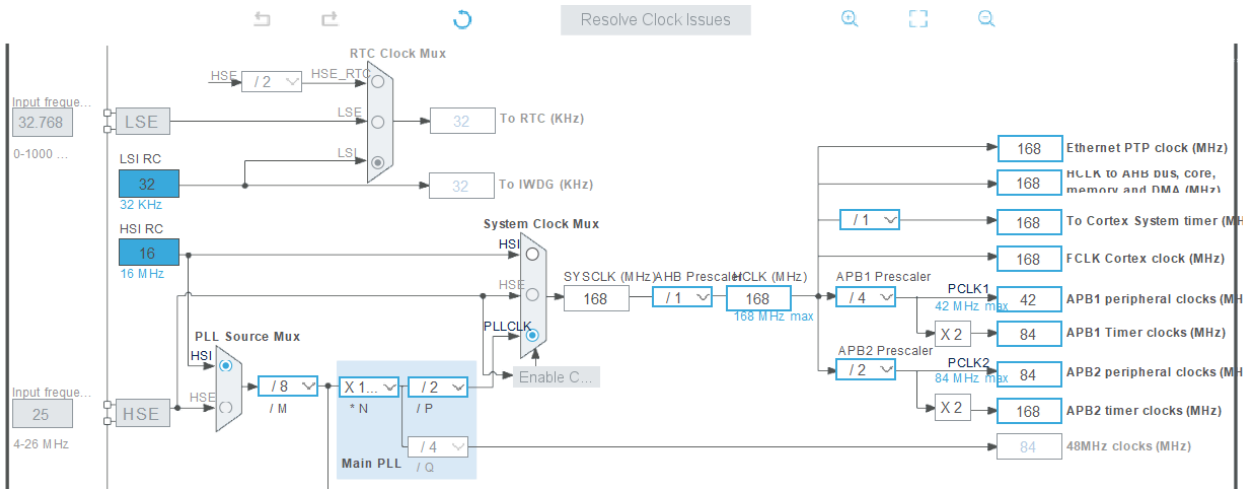
USART1 en mode transmission/réception : Baud rate = 115200





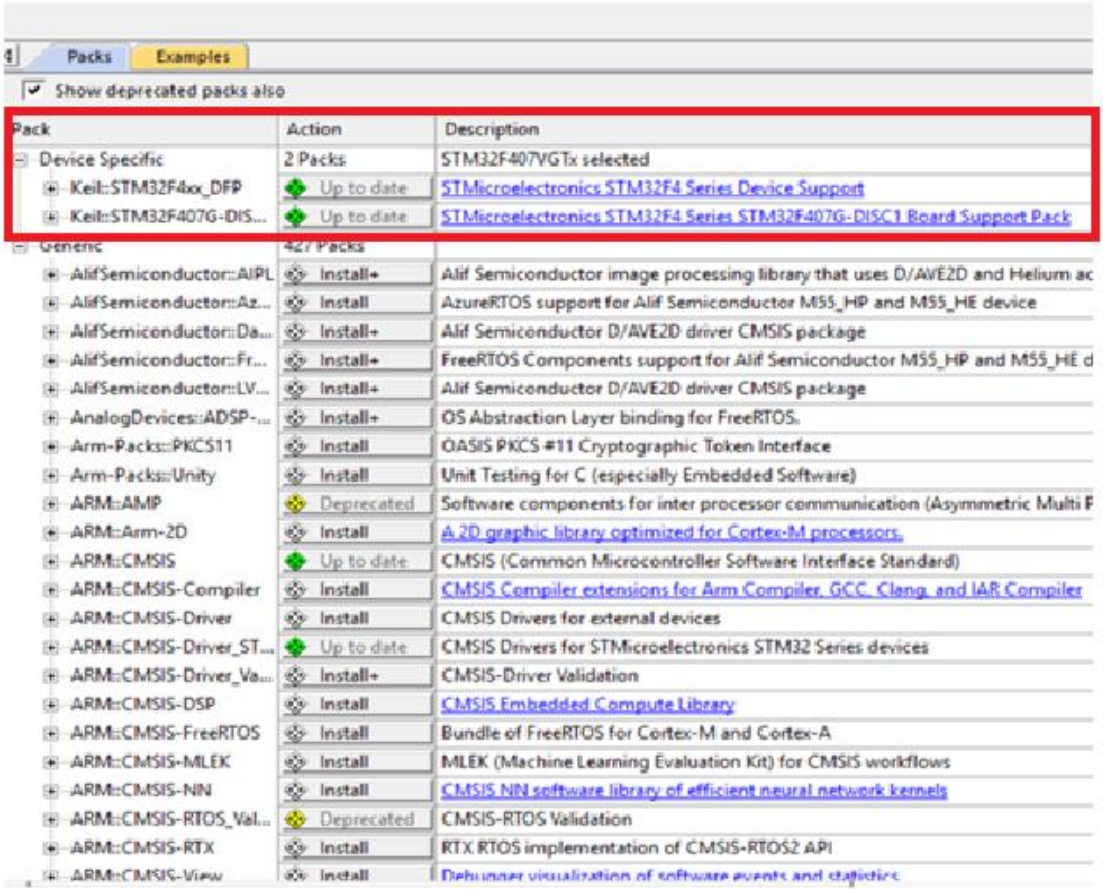
Le contrôleur d'interruptions NVIC est activé afin de permettre la gestion des interruptions liées aux périphériques CAN1 et USART1. En particulier, l'interruption de réception CAN est utilisée pour recevoir les trames CAN de manière asynchrone. Cette configuration permet l'exécution du code C développé, basé sur des callbacks d'interruption, garantissant une réception efficace des trames sans recours au polling.

**Configuration de l'horloge système à 168 MHz : **



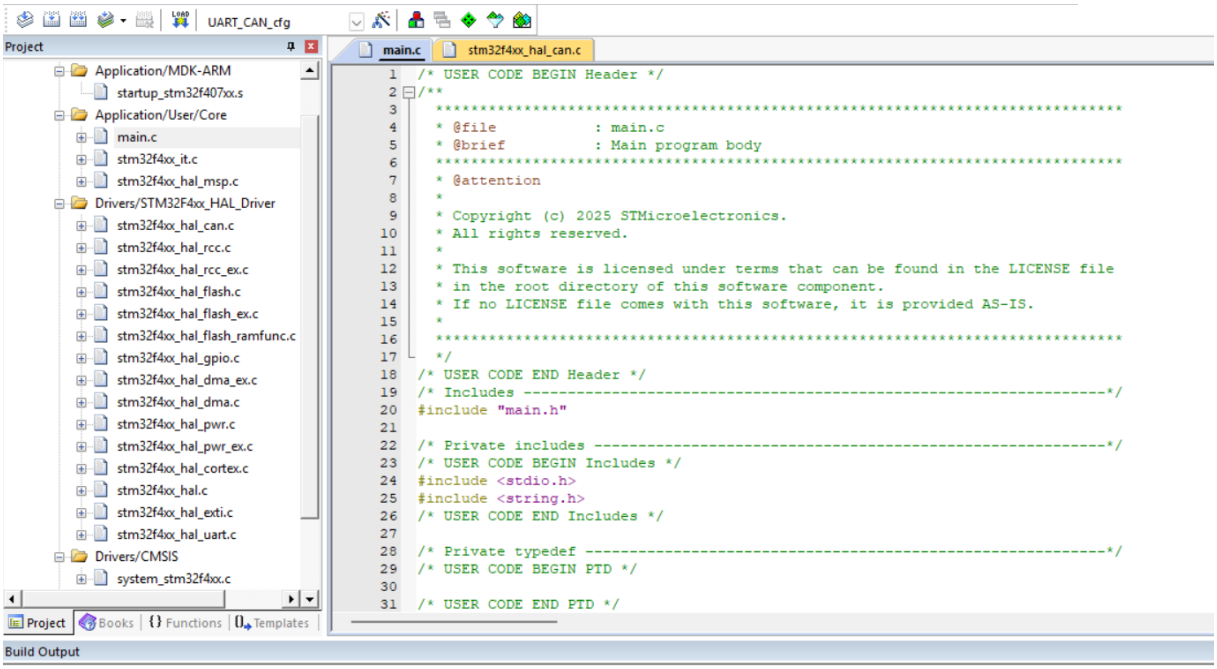
Le projet généré a ensuite été exporté vers Keil pour le développement.

4- Registres et configuration CAN/UART (Question 3)



L'interface obtenue à l'ouverture du logiciel Keil pour compléter le script du code C pour compléter le schéma du projet microcontrôleur STM CAN→USART→PC.

Dans Keil il est indispensable de choisir la carte et d'installer les packs associés à la carte STM32F407VGTx avec la bibliothèque HAL et établir l'espace d'application



Les fichiers à gauche représentent la configuration intérieure de la carte microcontrôleur

Et suivant ce qu'on avait activé de périphériques dans STM32CubeMX, on trouve toutes leurs méthodes et : sur le simulateur de cartes STM32CubeMX, on a activé 2 broches CAN1 et USART1.

Registre RCC :

Dans le main on trouve le registre d'horloge RCC dans la fonction SystemClock_Config()

le RCC active la source HSI de fréquence 16Mhz et configure le PLL, ensuite ce dernier produit SYSCLK = 168 MHz.

```
void SystemClock_Config(void)
{
  RCC_OscInitTypeDef RCC_OscInitStruct = {0};
  RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
  /** Configure the main internal regulator output voltage
  */
  __HAL_RCC_PWR_CLK_ENABLE();
  __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE1);
  /** Initializes the RCC Oscillators according to the specified parameters
  * in the RCC_OscInitTypeDef structure. */
  RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
  RCC_OscInitStruct.HSIState = RCC_HSI_ON;
  RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
  RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
  RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSI;
  RCC_OscInitStruct.PLL.PLLM = 8;
  RCC_OscInitStruct.PLL.PLLN = 168;
  RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV2;
  RCC_OscInitStruct.PLL.PLLQ = 4;
  if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
  {
    Error_Handler();
  }
```

Ici, on remarque que SYSCLK se divise pour alimenter les périphériques d'horloge

```
RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                            |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1; // HCLK = 168 MHz
RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV4; // PCLK1 = 42 MHz
RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV2;  // PCLK2 = 84 MHz

if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_5) != HAL_OK)
{
  Error_Handler();
}
```

Registre CAN1 :

Le registre CAN1 dans la fonction MX_CAN1_Init()

CubeMX génère une initialisation HAL qui configure les registres CAN internes : bit timing, mode, options

Ces paramètres configurent la durée d'un bit CAN en Time Quanta appelé TQ dans le but d'obtenir un bitrate proche du besoin réseau de 500 kbps (400-500kbps)

```
227 static void MX_CAN1_Init(void)
228 {
229
230
231   /* USER CODE END CAN1_Init 1 */
232   hcan1.Instance = CAN1;
233   hcan1.Init.Prescaler = 6;
234   hcan1.Init.Mode = CAN_MODE_NORMAL;
235   hcan1.Init.SyncJumpWidth = CAN_SJW_1TQ;
236   hcan1.Init.TimeSeg1 = CAN_BS1_13TQ;
237   hcan1.Init.TimeSeg2 = CAN_BS2_2TQ;
238   hcan1.Init.TimeTriggeredMode = DISABLE;
239   hcan1.Init.AutoBusOff = DISABLE;
240   hcan1.Init.AutoWakeUp = DISABLE;
241   hcan1.Init.AutoRetransmission = DISABLE;
242   hcan1.Init.ReceiveFifoLocked = DISABLE;
243   hcan1.Init.TransmitFifoPriority = DISABLE;
244   if (HAL_CAN_Init(&hcan1) != HAL_OK)
245   {
246     Error_Handler();
247   }
248   /* USER CODE BEGIN CAN1_Init 2 */
249
250   /* USER CODE END CAN1_Init 2 */
251
```

Le filtrage CAN est réalisé au niveau des registres de filtrage du contrôleur CAN : IDMASK et échelle 32 bits. Cette configuration permet au matériel de comparer directement les bits de l'identifiant standard StdID = 0x0000 et de ne placer en FIFO que les trames correspondant au masque configuré. Le traitement est ainsi effectué en amont par le périphérique CAN, réduisant l'activité du CPU et le nombre d'interruptions générées.

```
static void CAN_Config(void)
{
    CAN_FilterTypeDef sFilterConfig;

    sFilterConfig.FilterBank = 0;
    sFilterConfig.FilterMode = CAN_FILTERMODE_IDMASK;  // Mode "IDMASK" : on filtre avec un identifiant + un masque
    sFilterConfig.FilterScale = CAN_FILTERSCALE_32BIT;

  // Accepte que StdID 0x000 : ces trames correspondent a celle envoyés par arduino
    // pour un filtre 32-bit : StdID est décalé a gauche de 5
    sFilterConfig.FilterIdHigh     = (0x000 << 5);
    sFilterConfig.FilterIdLow      = 0x0000;
    sFilterConfig.FilterMaskIdHigh = (0x7FF << 5);   // correspond aux 11 bits de StdID
    sFilterConfig.FilterMaskIdLow  = 0x0000;

    sFilterConfig.FilterFIFOAssignment = CAN_RX_FIFO0;
    sFilterConfig.FilterActivation = ENABLE;
    sFilterConfig.SlaveStartFilterBank = 14;

    // Appliquer la configuration du filtre au périphérique CAN
    if (HAL_CAN_ConfigFilter(&hcan1, &sFilterConfig) != HAL_OK) Error_Handler();
    if (HAL_CAN_Start(&hcan1) != HAL_OK) Error_Handler();
    if (HAL_CAN_ActivateNotification(&hcan1, CAN_IT_RX_FIFO0_MSG_PENDING) != HAL_OK) Error_Handler();
}
```

Registre UART1 :

L'UART1 est configurée en 115200 bauds pour transmettre les trames au PC sous un format lisible

La connexion CAN-UART nécessite aucune des options secondaires : une simple connexion le CAN est actif seulement si des trames sont envoyés depuis le shield MCP2515

```
static void MX_USART1_UART_Init(void)
{

  /* USER CODE END USART1_Init 1 */
  huart1.Instance = USART1;
  huart1.Init.BaudRate = 115200;
  huart1.Init.WordLength = UART_WORDLENGTH_8B;
  huart1.Init.StopBits = UART_STOPBITS_1;
  huart1.Init.Parity = UART_PARITY_NONE;
  huart1.Init.Mode = UART_MODE_TX_RX;
  huart1.Init.HwFlowCtl = UART_HWCONTROL_NONE;
  huart1.Init.OverSampling = UART_OVERSAMPLING_16;
  if (HAL_UART_Init(&huart1) != HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN USART1_Init 2 */

}
```

- ⇨ La configuration des horloges via RCC fournit PCLK1 = 42 MHz, condition nécessaire pour un CAN stable. Le périphérique CAN1 est ensuite configuré en mode normal avec un bit timing adapté. La réception est gérée efficacement par interruption FIF0 (le CAN ne doit pas resté actif en perméance pour la réception des trames)

- ⇨ l'UART1 est configurée en 115200 bauds pour transmettre les trames au PC
  L'étape finale consiste à réceptionner les trames envoyées par le code sur Arduino par le CAN shield a l'UART qui transmet ces données au PC par le FTDI FT232RL USB

5- Écrivez un code pour recevoir des trames CAN BUS depuis Arduino :

On joue maintenant sur le fichier main du programme comme toute la configuration a créé les registres, la mise en place du fonctionnement correct et l'activation des périphériques CAN et USART sur STM32CubeMX

On commence par déclarer nos variables globales dans le fichier main

```
#include <stdio.h>
#include <string.h>
CAN_HandleTypeDef hcan1;
CAN_TxHeaderTypeDef   TxHeader;
CAN_RxHeaderTypeDef   RxHeader; //pin Can Rx responsable de la reception des trames
uint8_t               RxData[8];

UART_HandleTypeDef huart1;
```

CAN_HandleTypeDef hcan1;

C'est la "structure centrale" qui contient la config, l'état, les registres associés.

Toutes les fonctions HAL du CAN prennent un pointeur vers ce handle.

uint8_t RxData[8];

Buffer de réception de 8 octets max

CAN_RxHeaderTypeDef RxHeader;

Structure de header pour l'émission CAN

```
203 void CAN_SendFrameToPC_UART(CAN_RxHeaderTypeDef *hdr, uint8_t *data)
204 {
205     char msg[128];
206     int len = 0;
207
208     // Header
209     len += snprintf(msg + len, sizeof(msg) - len,
210                     "ID:%031X DLC:%lu Data:",
211                     (unsigned long)hdr->StdId,
212                     (unsigned long)hdr->DLC);
213
214     // Data envoyée depuis l'arduino vers le can
215     for (uint8_t i = 0; i < hdr->DLC; i++)
216     {
217         len += snprintf(msg + len, sizeof(msg) - len, " %02X", data[i]);
218     }
219
220     len += snprintf(msg + len, sizeof(msg) - len, "\r\n");
221
222     // envoyé au PC par le FT232RL
223     HAL_UART_Transmit(&huart1, (uint8_t*)msg, len, 100);
224 }
```

Cette fonction convertit une trame CAN reçue en message texte lisible puis l'envoie au PC via UART

l'attribut msg est la ligne crée dans l'UART pour contenir le message en trames dans l'adresse DLC sous format lisible

les trames dans DLC sont traitées et lues dans la boucle for qui enregistre ces trames dans l'attribut **data[i]**

Dans La ligne 223 on remarque la fonction de HAL est appelée : elle permet d'envoyer réellement la chaîne vers le PC

```
void HAL_CAN_RxFifo0MsgPendingCallback(CAN_HandleTypeDef *hcan)
{
    if (hcan->Instance == CAN1)
    {
        if (HAL_CAN_GetRxMessage(hcan, CAN_RX_FIFO0, &RxHeader, RxData) == HAL_OK)
        {
            // RxHeader.StdId = received ID
            // RxHeader.DLC   = number of bytes
            // RxData[]       = payload bytes

          // send it to PC via UART : human readable message
            CAN_SendFrameToPC_UART(&RxHeader, RxData);
        }
    }
}
```

C'est une fonction appelée automatiquement par la HAL quand :

- Un message CAN arrive dans FIFO0
- Ce message sera retranscrit dans la structure de lecture RxHeader
- Envoyer au PC

```
/* Private function prototypes -----------------------------------------------*/
void SystemClock_Config(void);
static void MX_CAN1_Init(void);
void CAN_SendFrameToPC_UART(CAN_RxHeaderTypeDef *hdr, uint8_t *data);
static void MX_USART1_UART_Init(void);
static void CAN_Config(void);
/* USER CODE BEGIN PFP */
```

Déclare les fonctions avant main() sinon on tombe sur l'erreur si utilisées avant définition.

```c
int main(void)
{
  /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
  HAL_Init();

  /* Configure the system clock */
  SystemClock_Config();

  /* Initialize all configured peripherals */
  MX_CAN1_Init();
  MX_USART1_UART_Init();
  HAL_CAN_Start(&hcan1);
  HAL_CAN_ActivateNotification(&hcan1, CAN_IT_RX_FIFO0_MSG_PENDING);
  CAN_Config();

  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
  }
  /* USER CODE END 3 */
}
```

Finalement, on fait appel aux fonctions dans le main programme pour la compilation finale : HAL_Init() réinitialise les périphériques, SystemClock_Config() configure l'horloge système.

Ensuite, on active tous les periphériques : USART, CAN1 .
HAL_CAN_ActivateNotification(&hcan1, CAN_IT_RX_FIFO0_MSG_PENDING)active l'interruption de réception pour déclencher le callback quand une trame arrive.

CAN_Config() applique la configuration CAN + filtres + interrupts

6- Conclusion

Ce projet STM32 a permis de mettre en œuvre une démarche complète de développement embarqué, allant de la configuration des périphériques à la réalisation d'une application fonctionnelle. L'utilisation de la bibliothèque HAL a facilité l'initialisation du matériel et assuré une bonne portabilité du code. L'architecture basée sur les interruptions a rendu le système réactif et efficace, tout en gardant une structure logicielle claire. Une configuration rigoureuse et des tests progressifs ont été essentiels pour garantir le bon fonctionnement et la fiabilité de l'application.
La mise en place d'une architecture logicielle structurée a ensuite été essentielle. L'utilisation d'interruptions, notamment pour la réception CAN, a permis d'obtenir un système réactif et efficace, évitant le polling et réduisant la charge du processeur. La séparation claire entre l'initialisation, la configuration spécifique des périphériques et la logique applicative a amélioré la lisibilité, la maintenabilité et l'évolutivité du code.